Attention is all you need: reflection, implementation & experiments

Egor Agafonkin*, Gleb Gorin*

Moscow Institute of Physics & Technology, Moscow

Group 10

Introduction

"Attention is all you need" [1] impact on Neural Networks can't be overestimated. In this piece of work we will not remind common principles of work of the original Transformer, instead we will cover uneasy-to-comprehend moments about the Transformer's work process. And as well, we have deployed model that achieved the score of **24.14 BLEU** (DE $\rightarrow$ EN), which is comparable to the original Transformer's. Finally, we have conducted a bunch of experiments with its structure for the sake of confirmation of some of the original paper's claims, alongside with the testing of several hypotheses, originated in the process of reading and comprehension. You may review our repository [4] with code for the large model stable training, and for various experiments as well.

Tricky parts in the paper

Tokenization & embeddings

While tokenization is received by us from a certain tokenizator, embedinngs are learned parameters, which we increment by positional encodings and then optimize during training. Intuitively, "similar" tokens' embeddings vary insignificantly, so we have to make them "closer".

Attention

By matrices $\mathrm{Q}, \mathrm{K}, V$ via attention we obtain a new matrix X, which is interpreted as all Q, K and V on the next self-attention layer, and as Q on the next cross-attention layer while K, V are being the encoder output.

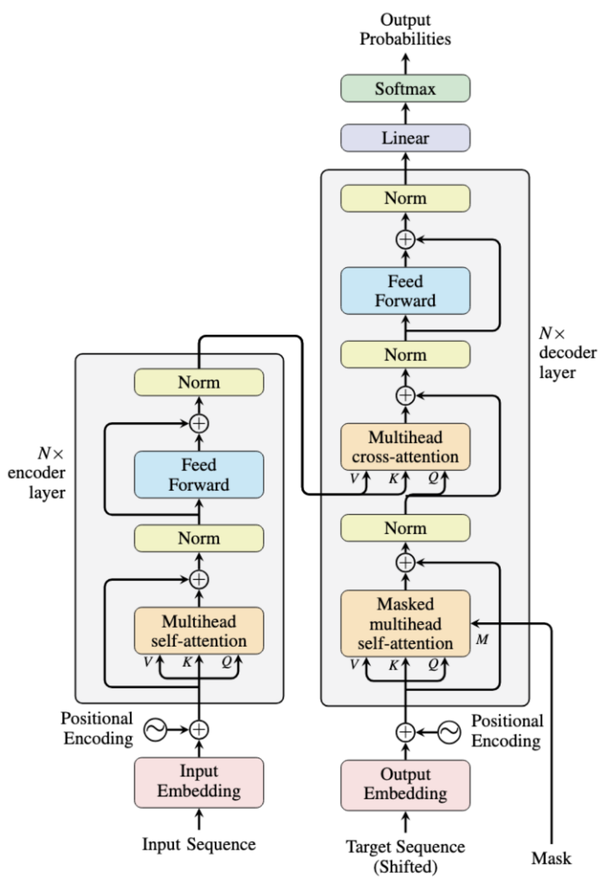


Figure 1: encoder-decoder mutual work [3]

In

$$\mathrm{MultiHead}(\mathrm{Q}, \mathrm{K}, V) = \mathrm{concat}(\mathrm{head}_1, ..., \mathrm{head}_\mathrm{h})W^\mathrm{O},$$

where

$$\mathrm{head}_\mathrm{i} = \mathrm{Attention}(\mathrm{Q}W_\mathrm{i}^\mathrm{Q}, \mathrm{K}W_\mathrm{i}^\mathrm{K}, VW_\mathrm{i}^V),$$

the formula

$$\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, V) = \mathrm{softmax}(\frac{\mathrm{Q}\mathrm{K}^\mathrm{T}}{\sqrt{\mathrm{d}_\mathrm{k}}}) \cdot V$$

is applied to matrices $\mathrm{Q}W^\mathrm{Q}, \mathrm{K}W^\mathrm{K}, VW^\mathrm{K}$, where $W^\mathrm{Q}, W^\mathrm{K}, W^\mathrm{V}$ are trained weight matrices, so the attention formula applied **not** to the output matrix from the past layer directly, but to the weighted outputs.

Encoder and decoder outputs meaning

Intuitively, on its output encoder gives you a contextualized matrix of tokens connectivity, their pairwise relations.

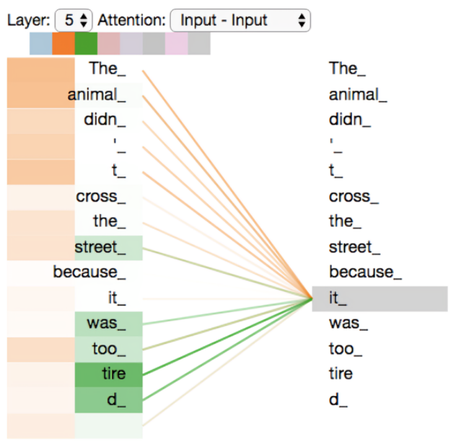


Figure 2: visuals for output of the encoder [2]

However, decoder's output is not that simple. One may assume, that it produces the probabilities of all vocabulary tokens to occur in the output, but it is slightly wrong: the decoder itself produces a matrix, which then is influenced by a linear and softmax layers, giving us the desired probability table.

Figure 3: visuals for output of the transformer [3]

Implementation

Logic via PyTorch

Algorithm 1: Scheme of the PyTorch based Transformer

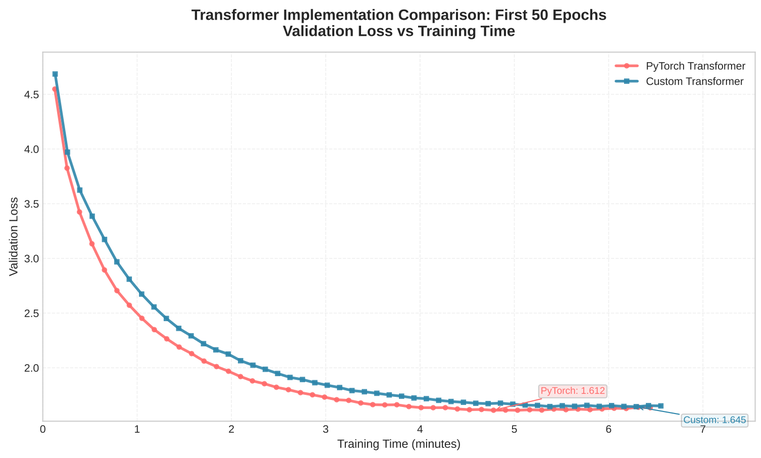
```
import torch.nn as nn
class Seq2SeqTransformer:
    self.transformer = nn.Transformer
transformer = Seq2SeqTransformer
train(transformer)
inferences...
```

Handwritten logic

We deployed our logic avoidng $\mathrm{nn.Transformer}$.

Performance comparison

They work practically similarly.



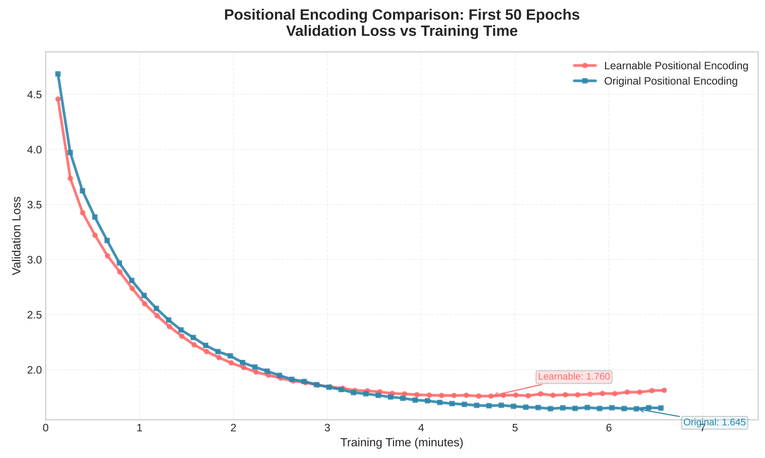
Experiments

Preamble

If not stated explicitly, we set $|\mathrm{Train}| = 30000, \mathrm{D}_\mathrm{MODEL} := 256, \mathrm{N}_\mathrm{LAYERS} := 3, \mathrm{N}_\mathrm{HEADS} := 4, \mathrm{D}_\mathrm{FF} := 512, \mathrm{DROPOUT} := 0.2, \mathrm{EPOCHS} := 50, \mathrm{LEARNING\_RATE} := 0.0001$ and original positional encodings.

Positional Encoding

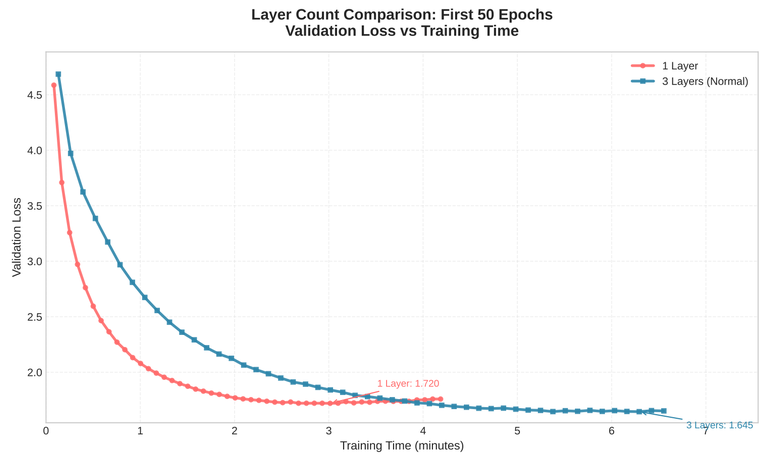
Instead of using precalculated constant positional encodings, we can use parametrized (adaptive).



With learnable, best loss is obtained during first epochs, and then it fits for the conventional word order within the sentence, (i. e., articles *"a", "an", "the"* commonly appear in the beginning of the sentence) so it accumulates certain bias, leading to overfitting.

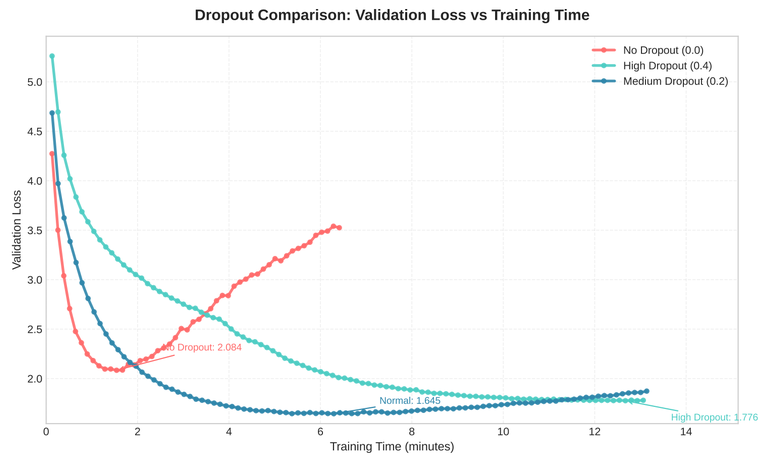
Layer variation

We try $\mathrm{N}_\mathrm{LAYERS} := 1$, and observe that the $3-$layered version is slower than $1-$layered, but it has more depth, so it learns quite more profound:



Dropout rate trying

Optimal dropout rate for us is 0.2, lesser dropout is bad since the dataset is "small" w. r. t. model, and higher dropout is bad the model is "small" w. r. t. dataset.



Large model rates

Our ultimate model was trained for 7.5 hours on **NVIDIA A100 Tensor Core GPU 80GB**, has 101M parameters, $\mathrm{train}_{size} = 2.000.000$, $\mathrm{vocabulary}_{size} = 37.000$ achieving **24.14 BLEU**.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin *"Attention Is All You Need"*

[2] https://jalammar.github.io/illustrated-transformer

[3] https://machinelearningmastery.com/encoders-and-decoders-in-transformer-models/

[4] https://github.com/sayheykid/transformers-ai360-practice

* – each author contributed equally.